

Full Stack Node.js Developer

• Delivery Window: 2 Days

Duration	2 calendar days	Expected Effort	10–14 hours
Mode	Individual	Submission	Git repository + README

1. Objective

Build a production-minded task management application that demonstrates your ability to design, implement, test, and document a complete full-stack solution using Node.js.

2. Project Scenario

Create a lightweight team task board where authenticated users can create projects, manage tasks, assign tasks to users, and track status changes.

3. Required Technology Stack

- Backend: Node.js with Express.js or NestJS.
- Database: PostgreSQL, MySQL, or MongoDB.
- Frontend: React, Next.js, Angular, or Vue.js.
- Authentication: JWT-based authentication with secure password hashing.
- API style: RESTful APIs with proper status codes and validation.
- Version control: Git with clear, meaningful commits.

4. Core Functional Requirements

Authentication & Users

- Register and log in users.
- Hash passwords securely.
- Protect authenticated routes.
- Provide at least two roles: Admin and Member.

Projects

- Create, view, update, and delete projects.
- Allow an Admin to add or remove project members.
- Show only projects accessible to the authenticated user.

Tasks

- Create, view, update, and delete tasks inside a project.

- Each task must include: title, description, status, priority, due date, creator, and assignee.
- Supported statuses: To Do, In Progress, and Done.
- Support filtering by status, priority, and assignee.
- Prevent unauthorized users from modifying projects or tasks they cannot access.

Frontend

- Provide login and registration screens.
- Provide a project list and task board or task table.
- Include create/edit forms with client-side validation.
- Handle loading, success, empty, and error states clearly.
- Use a responsive layout suitable for desktop and mobile.

5. Engineering Expectations

- Use a clear project structure and separation of concerns.
- Validate and sanitize request data.
- Implement centralized error handling.
- Use environment variables for secrets and configuration.
- Add database migrations or seed data where applicable.
- Write clean, readable, and maintainable code.
- Include at least 5 meaningful automated tests covering key backend logic or APIs.

6. Bonus Features

Bonus items are optional and should only be attempted after completing the core scope.

- Docker Compose setup for the application and database.
- API documentation using Swagger / OpenAPI.
- Real-time task updates using WebSockets or Socket.IO.
- Pagination, sorting, and search.
- Audit log for task status changes.
- Deployment to a publicly accessible environment.

7. Submission Instructions

1. Submit a public GitHub/GitLab repository or grant reviewer access to a private repository.
2. Include a complete README with setup instructions, architecture overview, environment variables, database setup, and test commands.
3. Provide a sample .env.example file without real secrets.
4. Provide API documentation or a Postman collection.
5. Include test credentials or seed instructions for Admin and Member accounts.
6. If deployed, include the live URL and any required access details.

DELIVERY DEADLINE

The completed task must be submitted within 48 hours from the time it is received.

8. Evaluation Criteria

Evaluation Area	Weight	What We Review
Backend architecture & API quality	25%	Structure, validation, security, error handling
Frontend implementation & UX	20%	Usability, responsiveness, state handling
Database design	15%	Schema quality, relationships, migrations
Code quality	15%	Readability, maintainability, conventions
Testing	10%	Coverage and meaningful test cases
Documentation & setup	10%	README clarity and reproducibility
Git practices	5%	Commit quality and repository organization

9. Reviewer Notes

- Candidates may make reasonable technical decisions where requirements are not explicitly stated.
- Incomplete bonus features will not compensate for missing core requirements.
- Use of AI development tools is permitted, but the candidate must understand and explain all submitted code.
- During the follow-up interview, the candidate may be asked to run the project, explain design choices, and implement a small change.